

AFP 项目教程

从零理解这个实验在做什么

写给新人——不讲目录，只讲概念和数据

2026 年 7 月 24 日

先讲一个故事

你有一个预训练好的语言模型 Pythia-1.4B。它的 13 亿个参数构成了一个起点。

现在你把它复制两份。一份喂代码推理数据训练，变成代码专家。另一份喂医学推理数据训练，变成医学专家。

训练结束后，两份模型的参数都变了。问题是：

1. 参数变了多少？这叫权重散度 (ΔW)。
2. 变化之后，它们还“挨得近”吗？这用 LMC 障碍来衡量。

直觉上，你会觉得：代码专家和医学专家应该走得挺远吧？毕竟学的东西完全不同。

但实验结果不是这样。决定两个模型“离多远”的，不是它们学了什么领域，而是这个领域本身的训练稳不稳定。

1 第一个概念：权重散度 ΔW

权重散度就是：训练后参数相比训练前，偏移了多少。

如果把模型的 13 亿个参数想象成 13 亿个坐标点，预训练模型是一个位置，微调后的模型是另一个位置。权重散度 ΔW 就是这两个位置之间的欧几里得距离，再除以预训练模型本身的“大小”（范数），得到一个百分比。

$$\Delta W = \frac{\|\theta_{\text{训练后}} - \theta_{\text{预训练}}\|_2}{\|\theta_{\text{预训练}}\|_2} \times 100\%$$

通俗理解： $\Delta W = 1.4\%$ 就是说，模型的参数平均偏移了 1.4%。

真实数据长什么样

我们训练了 4 组模型，每组 3 个不同随机种子（种子不同 = 训练数据的顺序不同）：

训练强度	Code 模型的 ΔW	Medical 模型的 ΔW	Code 和 Medical 之间的
标准（学习率 10^{-4} ）	1.4%	1.5%	2.0%
高散度（学习率 5×10^{-4} ）	8.0%	8.5%	11.6%

表 1: 权重散度实测值。来源：论文 §4.1。

关键观察：标准训练下，模型只挪了 1.4–1.5%。就算刻意加大学习率（高散度条件），也只挪了 8%。这说明微调对参数的影响远比你想象的小——模型基本上没走远。

还有一个重要细节：哪些层的参数变得多？

我们把 24 层 Transformer 的每一层分别计算 ΔW ，发现：

- 早期层（0–7 层）变得最多， ΔW 达到 5–6%。
- 后期层（16–23 层）几乎没变， ΔW 不到 1%。
- Code 和 Medical 的变化模式几乎一模一样，相关度 $r = 0.995$ 。

意思就是：微调主要改的是前面的层（特征提取层），后面的层（分类/输出层）基本不动。而且不管训练什么领域，改的都是同一批层——只是幅度略有不同。

2 第二个概念：LMC 障碍

LMC 障碍衡量的是：两个模型之间画一条直线，这条直线上 loss 最高点比两端平均高多少。

想象 loss 地貌是一张有山有谷的地图。两个微调后的模型是地图上的两个点。我们在它们之间画一条直线，在这条直线上均匀取 11 个点（就是 11 个插值模型），每个点都测一下 loss。

如果两个模型在同一个“山谷”里，这条直线的 loss 应该接近两端平均——障碍 ≈ 0 。如果它们之间隔着一座山，直线经过山顶时 loss 会很高——障碍 > 0 。

$$\text{障碍} = \max_{\alpha} \mathcal{L}(\theta(\alpha)) - \frac{\mathcal{L}(\theta(0)) + \mathcal{L}(\theta(1))}{2}$$

其中 $\theta(\alpha) = (1 - \alpha)\theta_{\text{code}} + \alpha\theta_{\text{medical}}$ ， α 从 0 到 1。

真实数据长什么样

跨域测量（code 模型和 medical 模型之间）：

条件	在 Code 测试集上的障碍	在 Medical 测试集上的障碍
标准散度 ($\Delta W \approx 2\%$)	0.053	0.051
高散度 ($\Delta W \approx 12\%$)	0.118	0.228

表 2: 跨域 LMC 障碍实测值。来源：论文 §4.2。

关键观察：

- 标准训练下，障碍只有 0.05——**几乎是一条平路**。code 和 medical 模型虽然在两个不同领域训练的，但它们还在同一个 loss 盆地里。
- 高散度下障碍翻了 2-5 倍，但依然不大。即使把权重推了 8%，模型之间仍然没有出现一座大山。
- Medical 在自身测试集上的障碍（0.228）明显比 Code 高——**这里埋着整篇论文最重要的发现**。

3 第三个概念：域内障碍 vs. 跨域障碍

前面测的是 code 模型和 medical 模型之间的障碍——这是跨域障碍。但我们还可以测域内障碍：两个同一个领域、不同随机种子的模型之间的障碍。为什么测这个？因为如果同域障碍本身就很高，那跨域障碍高就不奇怪——不是因为领域变了，而是因为这个领域的训练本身就不稳定。

真实数据

域内配对	障碍
Code 种子 0 对 Code 种子 1	0.048
General 种子 0 对 General 种子 1	0.071
Math 种子 0 对 Math 种子 1	0.087
Medical 种子 0 对 Medical 种子 1	0.147
参考：Code 和 Medical 之间的跨域障碍	0.053

表 3: 域内障碍实测值（标准散度）。来源：论文 §4.3。

这里就是整篇论文最核心的发现。

Code 模型的域内障碍 (0.048) 和跨域障碍 (0.053) 差不多。两个 code 模型之间的“距离”，和一个 code 模型与一个 medical 模型之间的“距离”是同一量级。领域差异? 对 code 来说不存在。

但 Medical 模型的域内障碍 (0.147) 是跨域障碍 (0.051) 的 **3 倍**! 两个在同一份数据上训练的 medical 模型，它们之间的 loss 地貌距离，比一个 code 模型和一个 medical 模型还要远。

结论: 决定障碍的不是“领域不同”，而是“这个领域训练起来有多不稳定”。Medical 数据就是比 Code 数据更不稳定。

4 第四个概念：训练轨迹

前面测的都是训练完成后的模型之间。但我们还想知道：训练过程中，障碍是怎么变化的？

我们每训练 40 步就保存一个检查点，然后测量这个检查点和预训练起点之间的 LMC 障碍。这样就得到了一条轨迹曲线：

- X 轴：训练步数（训练了多久）
- Y 轴：当前模型和预训练模型之间的 LMC 障碍

真实数据

训练步数	Code 障碍	Medical 障碍
40	0.029	0.173
80	0.027	0.163
120	0.037	0.157
160	0.043	0.150
200	0.043 （峰值）	0.143
240	0.043	0.148
280	0.038	0.154
320	0.034	0.150
360	0.035	0.147
400	0.033（下降）	0.151

表 4: 训练轨迹数据。来源：论文 §4.6。

Code 呈倒 U 形：障碍先从 0.029 上升到 0.043（步 200），然后下降到 0.033（步 400）。尽管步 400 时权重偏离预训练模型更远了（ ΔW 更大），但障碍反而变小了。这说明 Code 模型在训练后期收敛到了一个“宽而平”的最小值，这个最小值反而离预训练起点更“近”（在连通性意义上）。

Medical 呈单调增长：障碍从 0.173 一路升到 0.218（步 320），然后稳定在 0.21 左右。从未下降。Medical 模型在训练后期没有回到预训练模型的“附近”——它发散到了一个更远、更窄的盆地。

一句话总结：Code 训练会“走回来”，Medical 训练不会。

5 其他实验：用来排除替代解释

论文还做了一系列对照实验，用来排除“可能是别的原因导致的”这种质疑。逐一说明：

实验一：高斯噪声校准

问：障碍是不是仅仅因为参数幅度变了？如果我对模型随机加噪声，让它也有 8% 的 ΔW ，会不会也有障碍？

答：不会。实验结果是：即使加 $\Delta W = 8\%$ 的随机高斯噪声，障碍也只有 0.014——几乎为零。而训练产生的 $\Delta W = 1.5\%$ 却有障碍 0.053。

含义：障碍来自权重变化的方向（结构化、任务对齐的更新），不是幅度。随机扰动不产生障碍，定向训练才产生。

实验二：分层选择性插值

问：障碍集中在哪些层？

答：只合并前 8 层（层 0-7）时，障碍是 0.040，占全部障碍的 75%。只合并中间 8 层（8-15）时，障碍只有 0.003。只合并后 8 层（16-23）时，障碍是零。

含义：如果你要做模型合并，深层可以直接平均，省心省力。障碍全在前面几层。

实验三：种子对兼容性

问：前面说 medical 域内障碍高（0.147），这是每个模型都高，还是某些模型对特别高？

答：我们训练了 5 个高散度 medical 模型，两两配对（总共 $C(5,2) = 10$ 对），测量每对的障碍。结果：最低的一对只有 0.072（紧密连接），最高的一对高达 **1.213**（灾难性断裂）。差了 15 倍。

更有意思的是，同一个种子 s4：和 s2 配对时障碍只有 0.080，和 s1 配对时障碍飙到 1.213。

含义：障碍不是“这个模型稳定、那个模型不稳定”，而是配对特定的。取决于哪两个种子撞到了一起。这进一步证明了论文的核心论点：决定障碍的，不是单个模型的性质，而是两个模型各自走过的训练轨迹之间的几何关系。

实验四：跨架构复制

问：这些发现是 Pythia 特有的吗？换个模型架构还会成立吗？

答：我们用 OPT-1.3B（不同架构、不同训练数据、不同 tokenizer）重做了核心实验。结果：OPT 的绝对障碍值比 Pythia 大 4–8 倍（架构本身的差异），但 medical/code 域内障碍比值仍然是 $3.6\times$ （对比 Pythia 的 $3.7\times$ ）。**相对排序被复制了。**

模型	Code 域内障碍	Medical 域内障碍	Medical/Code 比值
Pythia-1.4B (2 epoch)	0.063	0.231	$3.7\times$
OPT-1.3B (2 epoch)	0.251	0.896	$3.6\times$

表 5: 跨架构复制结果。来源：论文 §4.9。

6 论文是怎么解释这些现象的？（理论）

论文提出了一个理论框架，核心思想是：障碍取决于权重变化的方向和 *loss* 地貌的曲率是不是对齐的。

通俗理解：

- 想象 *loss* 地貌像一块凹凸不平的橡胶。有些方向很“硬”（*loss* 变化大），有些方向很“软”（*loss* 变化小）。
- 如果训练的权重更新恰好沿着“硬”方向走，就会撞出一个大障碍。
- 如果沿着“软”方向走，权重虽然挪了很多，但障碍不大。
- Medical 数据标签噪声大、推理步骤的正确性模棱两可，导致训练过程沿着更加“尖锐”的方向更新——于是障碍更高。
- Code 数据标签干净、逻辑明确，训练收敛到宽而平的盆地——障碍自然低，甚至训练后期还会下降（倒 U 形）。

已做的理论验证：

1. **延长 Medical 训练到 2 epoch：**障碍从 0.147 升到 0.182——不稳定性不随训练消失，说明是数据本身的性质。

2. **给 Code 加 15% 随机标签噪声**：障碍不变（仍然是 0.046）——简单的随机噪声不足以制造 medical 级别的不稳定。Medical 的不稳定可能来自结构化的模糊性（步骤对错的争议性），而不是随机错误。
3. **测量 Fisher 信息量（局部曲率）**：Code 模型的最小值曲率更高（ 1.44×10^{-8} ），Medical 模型曲率反而更低（ 5.76×10^{-9} ）。这说明障碍不是因为最小值本身有多“陡”，而是因为两个最小值之间的盆地分离距离更远。

7 怎么跑实验

```
# 设置环境（每次开终端都要先跑这三行）
export HF_ENDPOINT=https://hf-mirror.com
export HF_DATASETS_OFFLINE=1
export PYTHONUNBUFFERED=1
cd /home/jiayu/AFP
VENV=/home/jiayu/AFP/venv/bin/python3

# 训练一个模型（约 42 分钟）
$VENV -u scripts/train_agent.py --domain code --lr 1e-4

# 跑 LMC 扫描（约 15 分钟，需要先有 code 和 medical 两个模型）
$VENV -u scripts/lmc_barrier_scan.py

# 批量实验
bash scripts/phase1_batch.sh

# 查看状态
bash scripts/monitor.sh
```

训练后必须验证 $\Delta W > 0.1\%$ 。历史上 Bug 22 导致 base 模型（未训练）被当作“已训练”模型保存，所有 LMC 实验全部无效。每次训练后确认权重真的变了。

8 几个重要的禁区

- **绝对不要清理共享内存**（`ipcrm`、`/dev/shm` 等）——会永久断开 Tailscale 远程连接。
- **不要把学习率写成科学贡献**。LR 只是制造散度的工具，论文讲的是稳定性的故事。
- **不要提 AFP / IVN / gate**。那个方向已经废弃了。

- 不要用软链接管理模型。每个模型独立放在 `{domain}_lr{lr}_s{seed}/` 下。
- 不要信任缓存。改了 `MAX_LEN` 之类配置后，必须删旧缓存重新 tokenize。
- 不要用 Python list 存 token ID。用 tensor。Python int 比 tensor int64 大 3.5 倍，大数据集会 OOM。

9 现在还需要做什么

1. **GPT-Neo-1.3B 复制**（最高优先级）。脚本已有，结果未知。这是论文缺失的最后一块实验拼图。
2. **修复 LMC 扫描脚本中的障碍公式 Bug**。脚本算的是 $\max(L) - L[0]$ ，正确的 Frankle 公式应该是 $\max(L) - (L[0] + L[-1]) / 2$ 。论文里的数字是手动修正的，跑脚本会得到错误结果。
3. **删除 $lr=2e-4/3e-4$ 的假结果文件**。6 个文件是字节完全相同的副本，扫描从未真正跑过。

用 `xelatex ONBOARDING_GUIDE.tex` 可重新编译本指南。

完整工程文档见 `docs/internal/ENGINEERING.md`（23 条 Bug 记录）和 `CLAUDE.md`（AI 操作规则）。

仓库：<https://github.com/FauReam/AFP>